GT Lost and Found

DCD-6137

Saket Suman, Alexander Anderson, Dalton Rider, Siwan Yang, Andy Zheng

Client: Ann McClellan

Repository: https://github.com/sman11235/cs3803-lost-and-found

Table of Contents

Table of Contents 2
List of Figures 3
Terminology 4
Introduction 5
  Background 5
  Document Summary 5
Static System Architecture 6
  Diagram 6
  Description 6
Dynamic System Architecture 7
  Introduction 7
  Sequence Diagrams 8
UI Design and Heuristic Analysis 10
  Introduction 10
Data Design 15
  Introduction 15
  Entity Relationship Diagram 15
  File Use 16
  Data Exchange 16
  Security Considerations 16
Appendix 18
  Members 18

List of Figures

Figure 1 - Static System Diagram 6
Figure 2 - Sequence Diagram for Nearby Item Search. 8
Figure 3 - Sequence Diagram for Item Attribute Filtering. 9
Figure 4 - Map Feature displaying all lost items within a set radium 10
Figure 5 - Feature to log found items 11
Figure 6 - Item found after searching via filters 12
Figure 7 – users can find browse lost items location, post- date and current status 13
Figure 8 – User to User Messaging System 14
Figure 9 - ER Diagram 15

Terminology

API – A set of rules and protocols that allow different software applications to communicate, exchange data, and share functionality.

HTTP – An application-level transfer protocol designed for transferring hypermedia documents like HTML or JSON between browsers (clients) and web servers.

REST API – A specific type of stateless API that facilitates the transfer of resources over HTTP in a simple and scalable way.

Geolocation – Process of determining the geographic location of a device.

Backend – The part of the application that handles business logic and data persistence. Cannot be accessed by the user.

Frontend – The part of the application the user interacts with.

JSON (JavaScript Object Notation) – Text-based data exchange format.

SPA (Single Page Applications) - a web application that loads a single HTML page and dynamically updates the content using JavaScript.

React – A frontend, component-based framework designed to make SPAs.

N-Tier Architecture – a software design pattern where an application is separated into distinct logical layers and physical tiers, each running on a different machine.

Relational Database – a database which organizes information into tables (relations) with rows (records) and columns (attributes).

Spring Boot – A Java framework with built in tools designed to handle authentication, build REST APIs, and manage database interactions.

JWT (Json Web Token) Authentication – A method of authentication and authorization that uses cryptographic signatures and claims to verify the user's identity and permissions.

PostGIS – An offshoot of PostgreSQL (a relational database) that comes with built-in tools that geolocation and coordinate management.

WebSocket – A protocol that enables two-way data transmission in real-time.

OpenCV – A python library that provides functionality for Computer Vision (CV)

GCS – Google Cloud Storage, a cloud file storage system that enables serverless storage

Introduction

Background

Our application is a webapp that facilitates the sharing of information about lost items on the Georgia Tech campus. Our goal is to provide students with accurate information about lost items on campus and ensure that the items are returned to their rightful owners. Our plan is to create an application utilizing N-tier architecture, consisting of a user interface layer, a business logic layer, a data access layer, and a data persistence layer. For our user interface layer, we are using the React web framework, so our application can be accessed on demand from a web browser. For the business logic and data access layers, we are using Spring Boot, due to its numerous built-in features and existing ecosystems. Finally, for the data persistence layer, we will be using PostGIS, due to its native support for geolocation and coordinate systems. Our application includes functionality for posting lost/found items, chatting with finders, managing lost item inventory, and automatic image tagging with computer vision. Users will be able to search for lost items using map interfaces, as well as filterable list interfaces.

Document Summary

The Static **System Architecture** portion of the document will provide a high-level overview of the major components of the application – the user interface, business logic, data access, and data persistence layers – and define how they will interact with each other.

The **Dynamic System Architecture** portion of the document will provide an in-depth overview of the dynamic interactions between the low-level components of the application. Specific classes and methods that make up each component will be specified in detail.

The **Data Storage Design** portion of the document will define the database schema we will be using for this application. It will include an ER diagram for ease of viewing. It will also outline data transfer between the layers and security concerns.

The **UI Design** section presents and describes the primary user interface screens of our application.

Static System Architecture

In this section, we will define the interactions between the various components of our application.

Diagram

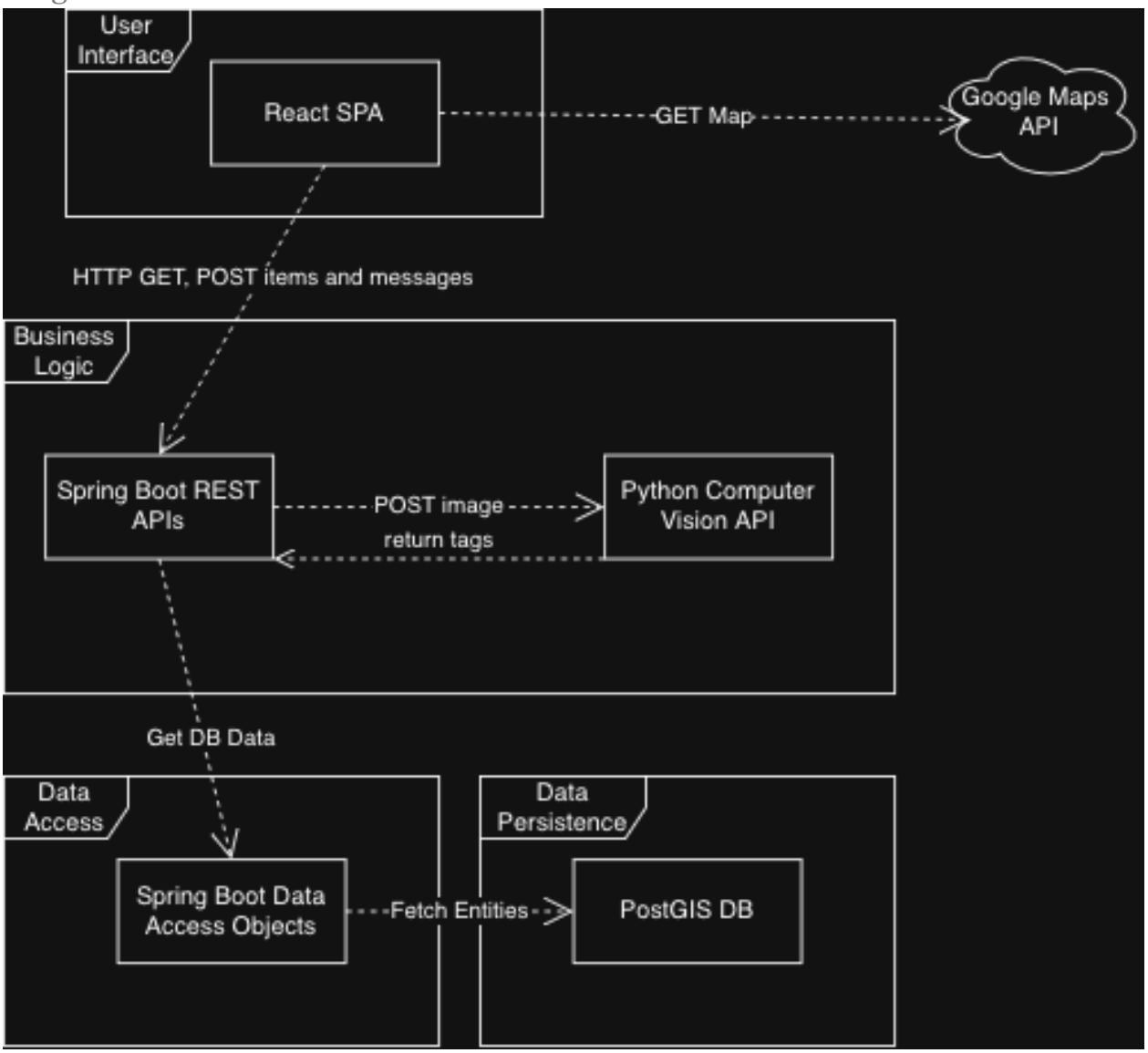


Figure 1 - Static System Diagram

Description

Our application can be broken down into 4 major subsystems, the UI layer, the business logic layer, the data access layer, and the data persistence layer.

Our UI layer utilizes the React framework. This is the industry standard when it comes to building a web deployed application and allows us to create an interactive SPA

frontend for our users to use. The UI layer aggregates and validates data from the user and sends it to the business logic layer for processing.

One layer down is the business logic layer. For this, we will have two components: the Spring Boot Central app and the Python Computer Vision API. The Spring Boot Central app will be the central heart of the application, managing posting, listing, and updating of lost items. It will also oversee both chat functionality and user authentication. It will accomplish this using REST APIs with Spring MVC and WebSocket libraries. The Spring Boot Central app will ingest user sent data from the UI layer and run validation to make sure the requested data is not malformed.

After the Spring Boot central app has finished processing the data, it will send the data to the Python Computer Vision API. This API ingests a picture the user sent and returns tags that will be assigned to the resultant entity that the Spring Boot Central app will save. It will use the OpenCV library for all matters related to computer vision.

Once the entities are properly constructed, they will be propagated to the Spring Boot Data Access Objects (DAOs), bringing us to the Data Access Layer. These objects will be connected to the PostGIS database using the Hibernate ORM. Hibernate is the industry standard for database access in Java.

The DAOs will send the fully constructed entities to our fourth and final layer, the data persistence layer. For this, we will be using PostGIS due to its native support for geolocation and latitude + longitude. This layer will be the source of truth for our application, and the final component in which all processed data will be stored.

Our application handles sensitive user data, so security is our utmost priority. However, user experience is also necessary. For this reason, we decided to include multiple methods of authentication. First, we implemented self-signed JWT-based authentication with a blacklist. This method gives us much more control over user permissions, and the ability to blacklist certain user tokens in case they are hijacked. Second, we implemented Google OAuth SSO. This method allows the user to forgo creating a new username and password and simply authenticate with their Gmail account. This makes creating an account and logging in is much easier and convenient.

Dynamic System Architecture

Introduction

While structural diagrams illustrate the static "what" of our system's architecture, dynamic diagrams illustrate the "how"—the runtime behavior and interactions between components. This section provides a view into the system's dynamic architecture through two UML Sequence Diagrams that depict core item-finding functionalities.

The first diagram (Figure 2) illustrates a non-trivial use case: searching for items within a specific geographic radius. It traces the flow from the user providing location data in the React frontend, through an API call to the Spring Boot backend, which in turn leverages the database's powerful PostGIS extension to perform an efficient spatial query. The second diagram (Figure 3) shows the more common flow of filtering the main item list by attributes like status or category. This sequence demonstrates how

user selections in the UI trigger a parameterized request to the backend, which then dynamically constructs a SQL query to fetch the precise subset of matching items.

Together, these diagrams reinforce the system's architectural integrity by showing how its decoupled components collaborate effectively to fulfill both simple and complex user requests.

Sequence Diagrams

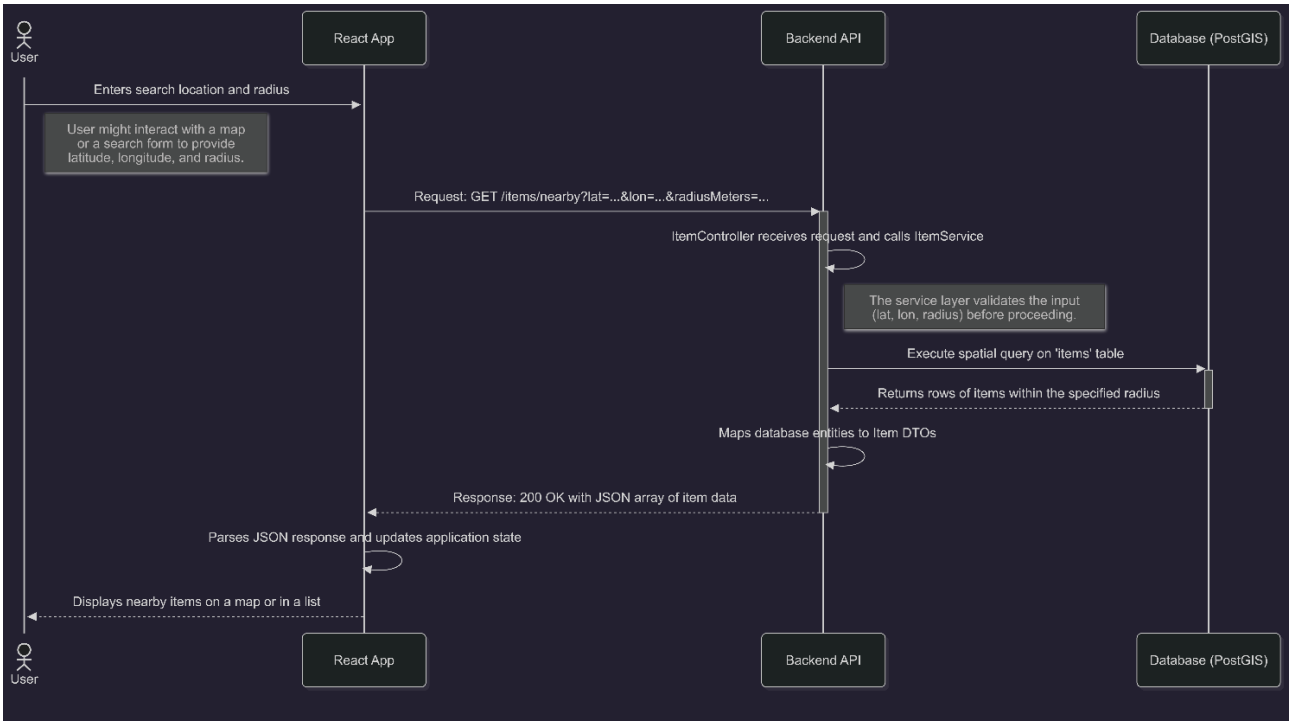


Figure 2 - Sequence Diagram for Nearby Item Search.

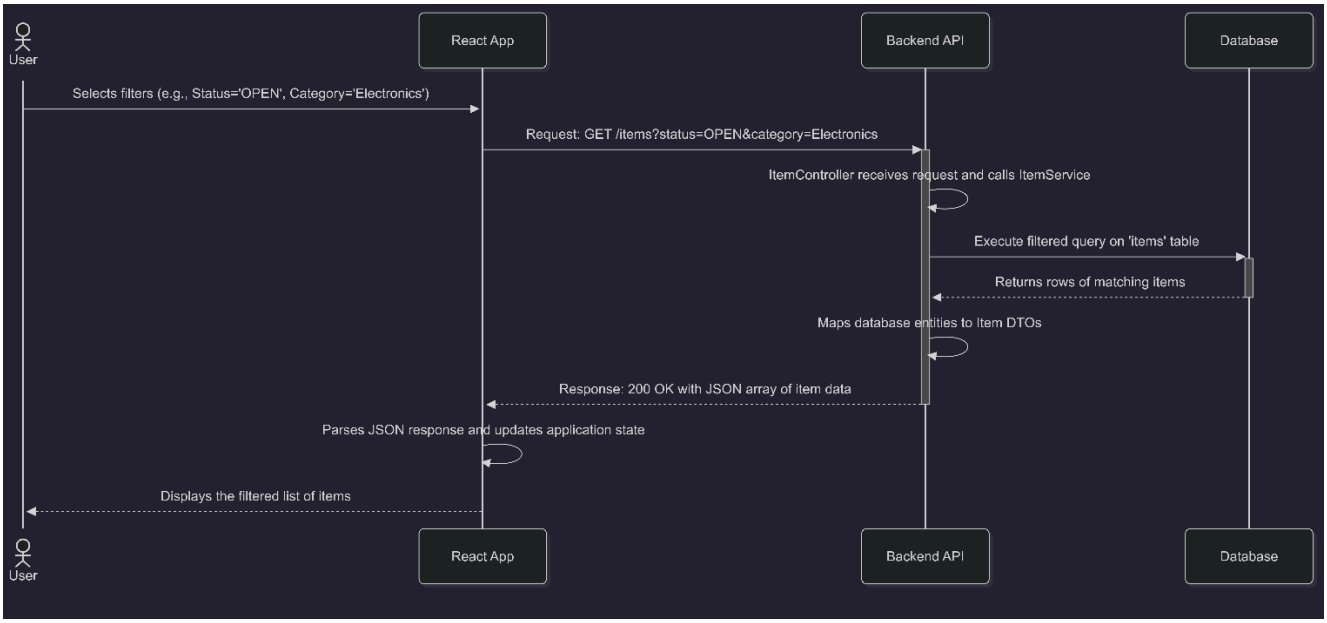


Figure 3 - Sequence Diagram for Item Attribute Filtering.

UI Design and Heuristic Analysis

Introduction

In the following section, we will explain the design choices and the corresponding heuristics that apply to them. The users of our application use the UI to interact with the backend of our application, and the goal is for the user experience to be smooth and convenient.

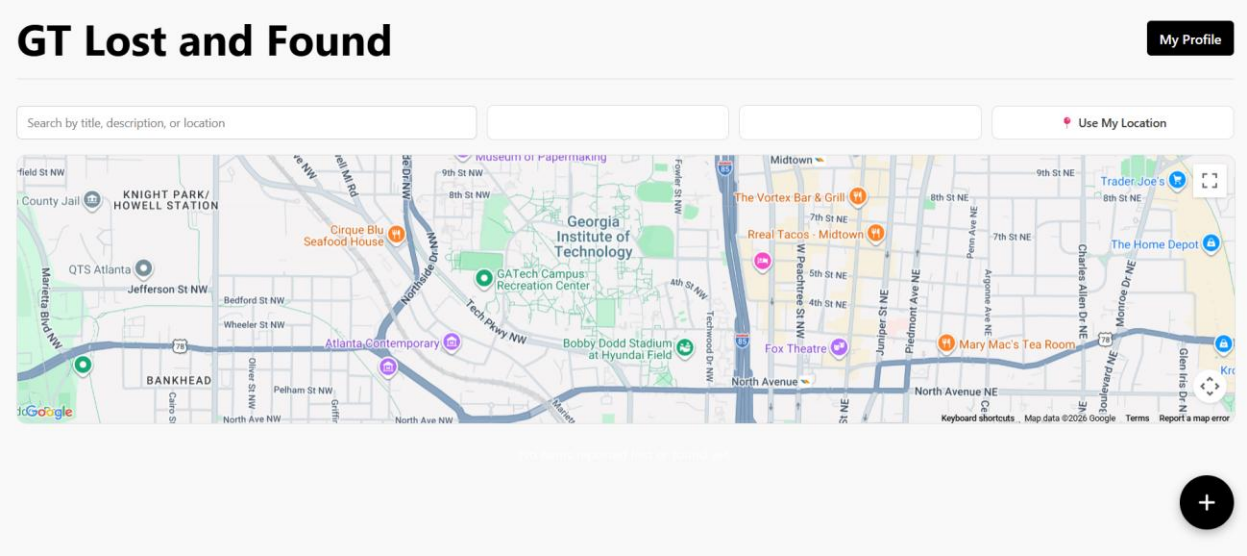


Figure 4 - Map Feature displaying all lost items within a set radium

When the user logs into the application, they are brought to the home page, as shown in figure 4. They have a map list display, a list display at the bottom, and a plus button that allows them to create a new post.

This page uses the following design heuristics: Clarity, Aesthetics and Minimalism, and User Control. Clarity because the main features of the application are front and center in the UI (the lost item posts and their locations, filtering options). Aesthetics and Minimalism because actions such as filtering and posting are represented with singular buttons that utilize iconography (such as the plus sign). And finally User Control because every part of the app is reachable from this page (the user profile, the list and map views, the filtration).

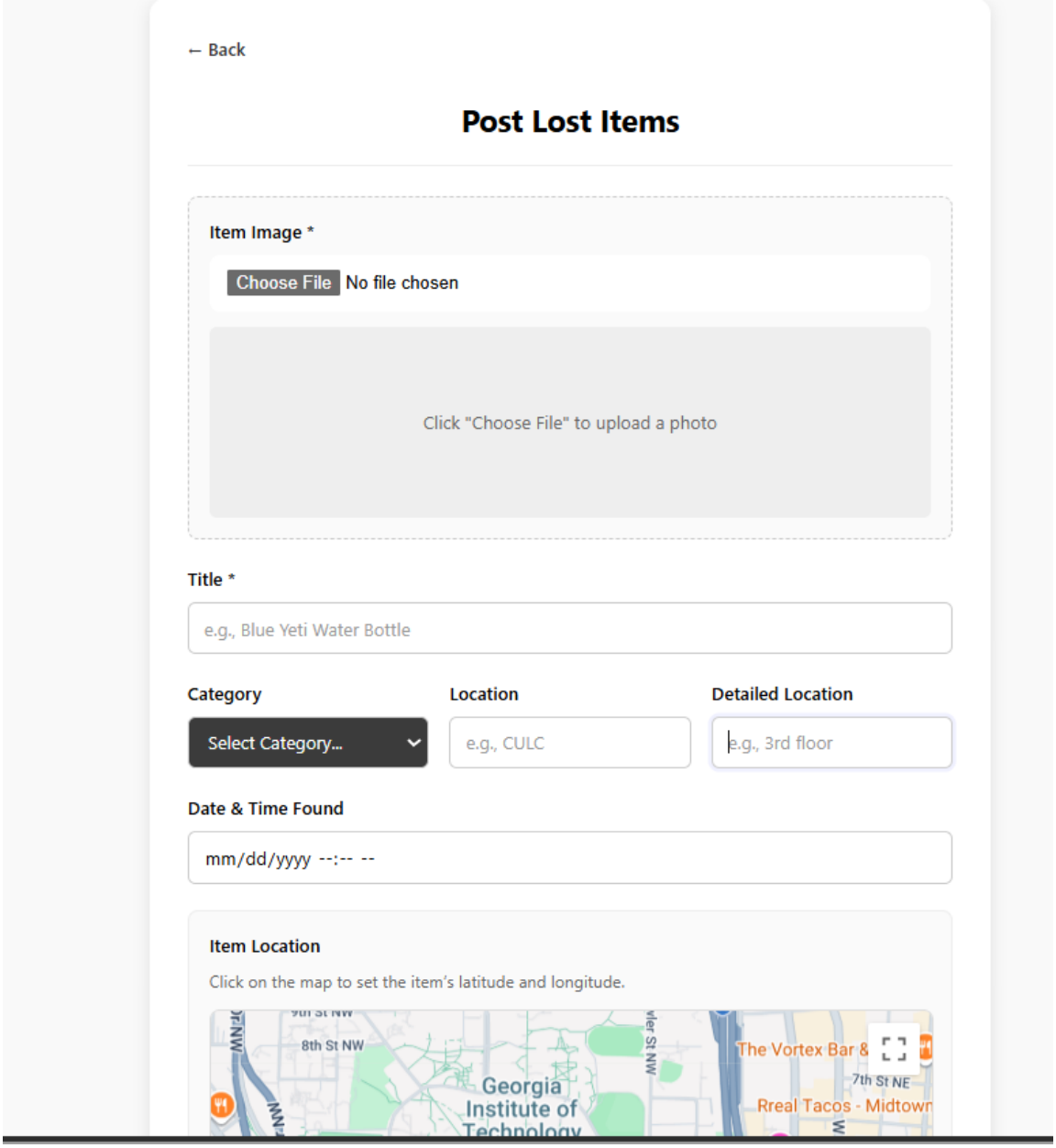


Figure 5 - Feature to log found items

Upon clicking the plus button, the user is brought to the post page. The post page offers users tools to fill out details about the lost item they picked up. These fields include a title for recognition, a location for the map view, and a picture for the item's post list view.

This page uses the following design heuristics: Error Prevention, Help and Guidance, and Error recovery. Error prevention because the location field and the location map picker are linked. If a location is picked in the location field, it will show up in the map below. This prevents discrepancies between the two location pickers. Help and Guidance

because each information field has a label, and uses automatic formatting (such as the date and time field enforcing a mm/dd/yyyy HH:MM:SS format). Finally, error recovery because if you enter an invalid field (such as a found date after the current date), the UI shows an error telling you to change the specific field.

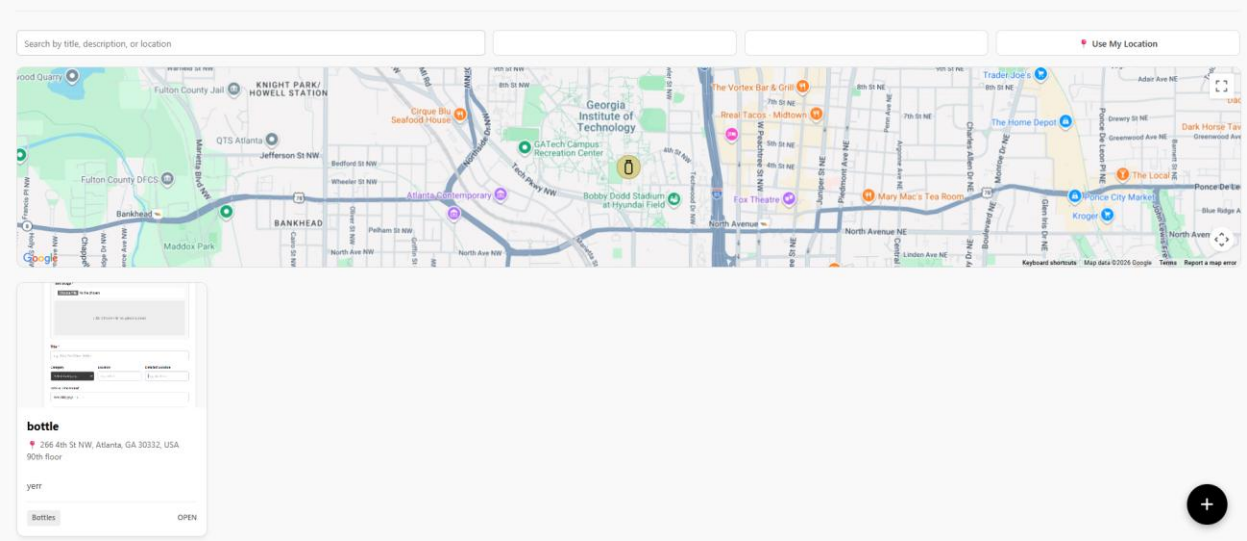


Figure 6 - Item found after searching via filters

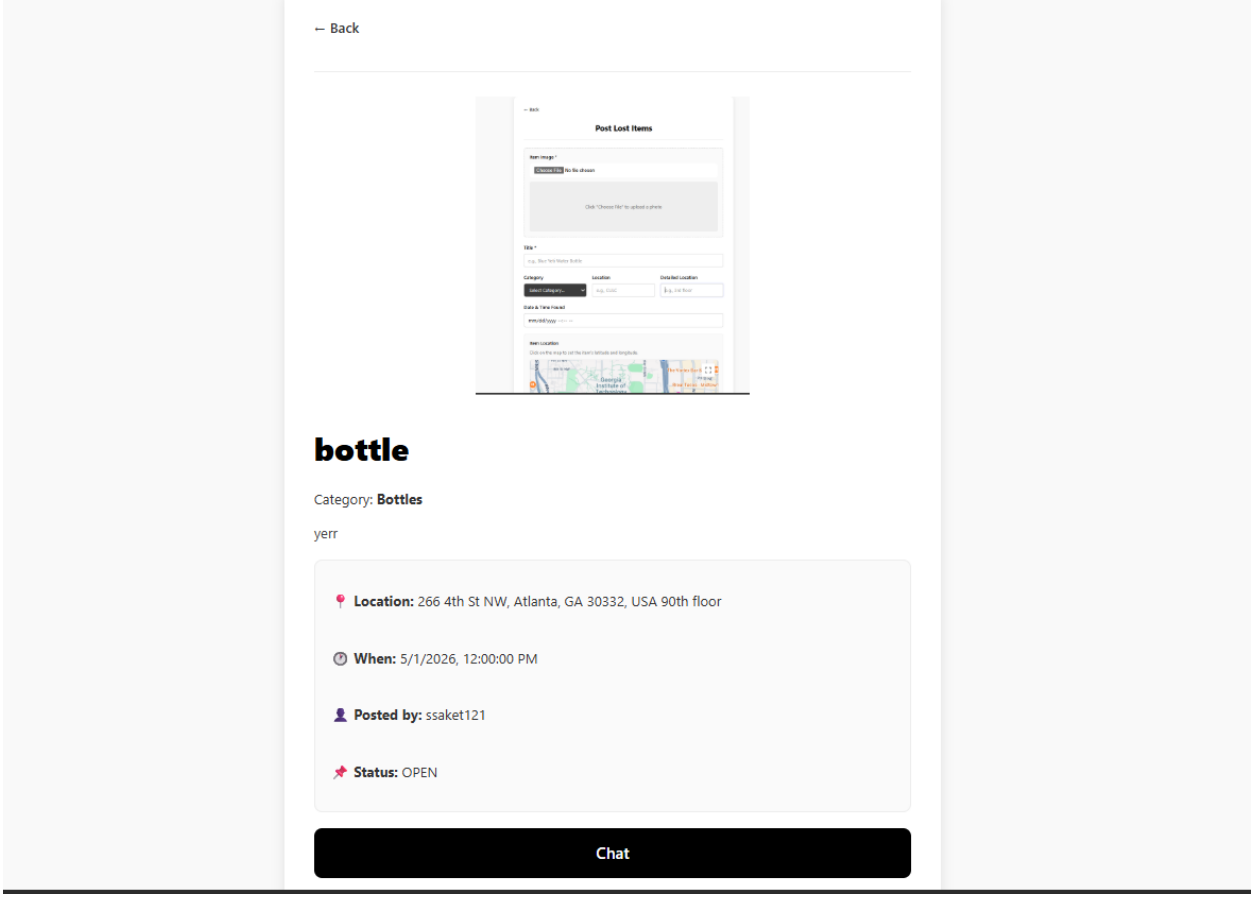


Figure 7 – users can find browse lost items location, post- date and current status

After locating an item on the home page that a user would like to learn more about, they click on the item post and are brought to this page. The user is presented with an image of the item, the category, description, location, date posted, finder/poster, as well as the item status.

The heuristics used in designing this page are as follows: Help and Guidance, Clarity, and Trustworthiness. Help and guidance are present in the fact that there is a repetition of important identifiers for a given lost item before a user can get to the messaging system. There is clarity in the amount and quality of information on the page. There are only key details mentioned that can help a user quickly verify that the item that they have clicked on is indeed theirs. Trustworthiness in build in the fact that important user verifiable identifiers are present, in the date that an item is posted, such that a user can fact check the time on it, and to build on this we added a chat feature that allows those who have found items to ensure to the best of their ability that the person claiming an item is the rightful owner.



Figure 8 – User to User Messaging System

This is the chats page that allows users that have lost an item to interact with the finder of the item to organize retrieval/ask questions about the items.

This page demonstrates a good understanding of the following heuristics: Aesthetic and minimalist design, clarity, and context preservation. The aesthetic and minimalist design can be seen in the simplistic navigation bar that allows those receiving messages to locate relevant chats. The clarity can be seen in the segmentation of the page, with there being a distinct section for the chat selection and then the main section of the page with the most important aspect, the chat between users. The context preservation can be seen in the fact that chats are stored and saved separately for different items. This allows users to understand what item they are being messaged about, and what messages so far have aligned with which item.

Data Design

Introduction

In figure 2, we have given an Entity Relationship (ER) diagram that corresponds to the needs of our application. This diagram shows the types of entities, attributes, and relationships between entities. We chose to use PostGIS for our database.

Entity Relationship Diagram

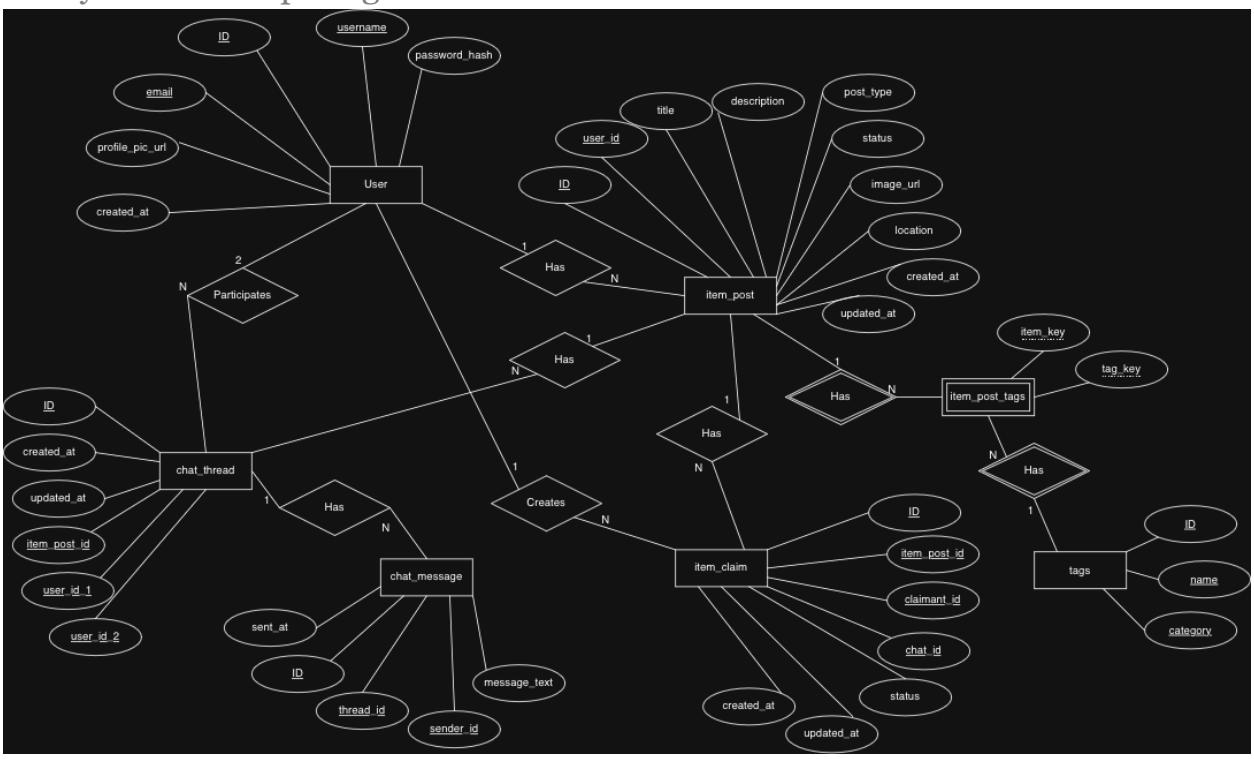


Figure 9 - ER Diagram

Here is a list of all the major entities:

1. User – This entity represents the user of the application. Most features of the application are tied to the user. As such, there are several relationships connected to it.
2. item_post – This entity represents a post the user makes. This is the object that will be displayed on the map and in list views of the frontend.
3. item_claim – This entity represents a claim on a lost item. There can be several claims on one item from different users. As such, there is a 1-to-many relationship between item_claim and item_post.
4. chat_thread – This entity represents a chat between users. As seen in the diagram, it is connected to item_post, since the chats are started when an item is claimed.
5. chat_message – This entity represents a message in a chat thread.
6. tags – This entity represents a tag. There will be a set number of tags for ease of filtering.

7. item_post_tags – This entity represents a tag applied to a post. It is a weak entity, whose only purpose is to connect tags to posts.

File Use

Our system supports the standard web-compatible image formats JPEG (.jpg, .jpeg) and PNG (.png). These files were selected due to their native browser support and compatibility with the Python Computer Vision API.

These image files are stored in text format in our PostGIS database. This is a temporary solution, and they would ideally be stored on a cloud server such as GCS (Google Cloud Storage) due to their large size, bloating the relational database. Future changes would allow the cloud URLs of the images to be stored in PostGIS, completely solving this issue.

When a user wants to generate tags for an item, the image is processed by the Python Computer Vision API to generate these tags. The image file will remain in cloud storage, and the URL and generated tags will be saved in the Data Persistence Layer.

Data Exchange

Our application uses a RESTful API over HTTP (HyperText Transfer Protocol to allow our React SPA and the Spring Boot backend to communicate with each other. The actual data that is being passed between the client and the server is formatted using JSON when sending and receiving requests, such as posting an item or retrieving the lost item inventory.

Real-time features such as the messaging system between users are handled with the WebSocket protocol. This allows for seamless two-way communication between users where messages can be exchanged without using HTTP.

Data exchange also occurs internally between the Spring Boot Central app and the Python Computer Vision API. The backend server will send image files via a POST request to the API, which uses the OpenCV library to process the image and return tags in a JSON format. These tags are then returned to the backend and stored in the PostGIS database.

Security Considerations

Our application will handle sensitive student data, so security is a primary focus. We have several strategies to improve overall security.

Account authentication will be implemented with two layers. We use JSON Web Tokens with a blacklist to manage user permissions and block any suspicious or stolen access tokens from being used again. Users are also able to authenticate their accounts using Google OAuth SSO, providing convenience in addition to security.

To provide security for our internal data exchange, all communication between the React SPA and Spring Boot will occur over HTTPS (Hypertext Transfer Protocol Secure). This way, our data will be encrypted when exchanged between the two.

Our application does collect Personally Identifiable Information such as usernames and contact information for the messaging system. This data will be stored in a PostGIS

database that is not accessible to the public and is restricted through the business logic layer.

The Spring Boot Central app will protect our data integrity by running validation on user-provided data to ensure that it is properly formatted before it is stored in our database.

Finally, our image assets ideally should be secured by GCS, which is a reliable and secure cloud storage system, but time limitations have led to us storing images in text format directly in PostGIS.

Appendix

Members

Alexander Anderson oversaw frontend development and construction of dynamic design diagrams. Contact is aanderson333@gatech.edu.

Saket Suman oversaw backend development and construction of static design diagrams. Contact is ssuman36@gatech.edu.

Andy Zheng oversaw the development of object recognition APIs. Contact is azheng88@gatech.edu.

Dalton Rider oversaw frontend development. Contact is drider7@gatech.edu.

Siwan Yang oversaw development of both frontend and backend portions of the project. Contact is syang723@gatech.edu.